

Multi-label Emotion Detection

Jovial Vernon Dsilva, Mohsin Ahmad, Palwinder Singh, Ali Heydari, Ram Binay Gupta

Phillips University Marburg

Marburg, Germany

{Dsilva, Ahmadmo, Singhpa, Heydari4, Gupta}@uni-marburg.de

Abstract

This paper presents a multi-label emotion detection system designed to identify five basic emotions (joy, sadness, fear, anger, and surprise) from text snippets. Each emotion is treated as a separate binary classification task, allowing for the simultaneous detection of multiple emotions. Our motivation stems from the significant applications of emotion detection in areas such as enhancing chatbot responses, tracking public sentiment for marketing and policy-making, and facilitating early intervention for mental health. We utilize a labeled dataset with binary indicators for each emotion. The methodology involves comprehensive preprocessing, exploration of various text representations (TF-IDF, Word Embeddings, and Transformer-based embeddings like BERT), and experimentation with different models, including Logistic Regression, Naive Bayes, Neural Networks, and fine-tuned Transformer-based models. Binary Cross-Entropy Loss is employed for the multi-label classification, incorporating ‘pos_weight’ to address class imbalance. Evaluation is performed using per-emotion metrics (precision, recall, F1-score) and aggregate metrics (micro-averaged F1, macro-averaged F1, Hamming loss, and Area Under the Precision-Recall Curve (AUPRC)). We address class imbalance through over-sampling and augmentation, and employ stratified n-fold cross-validation for hyperparameter tuning. Initial analysis of the dataset reveals an imbalance in emotion distribution and a predominance of short texts.

1 Introduction

Emotion detection in text is a crucial task in Natural Language Processing (NLP) that aims to identify and classify human emotions expressed in written language. Unlike traditional

sentiment analysis, which typically categorizes text into positive, negative, or neutral sentiments, emotion detection seeks a more nuanced understanding by identifying specific emotions. This project focuses on building a multi-label emotion detector capable of predicting the presence of one or more of five basic emotions: joy, sadness, fear, anger, and surprise, for a given input text snippet. Each emotion is treated as an independent binary decision (1 = emotion present, 0 = not present), enabling the model to assign any combination of these labels to a single text.

The motivation behind this project is multifaceted, driven by the significant real-world applications of accurate emotion detection. Firstly, in the realm of human-computer interaction, understanding user emotions in chatbots and virtual assistants can lead to more genuine, empathetic, and personalized responses, thereby improving user experience. Secondly, emotion detection can be instrumental in tracking public sentiments and emotional trends across various domains, such as crisis response, marketing campaigns, and social movements. This capability provides valuable insights for brands, policymakers, and researchers. Lastly, from a mental health perspective, identifying persistent patterns of sadness or anger in user communications could facilitate early intervention and care, offering a proactive approach to mental well-being. These compelling use cases underscore the importance and relevance of developing robust multi-label emotion detection systems. This work builds upon advancements in NLP by exploring the effectiveness of various models, including transformer-based architectures, for multi-label emotion detection.

2 Related Work

Emotion detection has been a long-standing research area in NLP, with various approaches ranging from lexicon-based methods to machine learning and deep learning models. Early works often relied on handcrafted rules and emotion lexicons to identify emotional words and phrases. With the advent of machine learning, models like Support Vector Machines (SVMs) and Naive Bayes classifiers were applied to this task, often using features like TF-IDF and n-grams. More recently, deep learning architectures, particularly Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), have shown promising results in capturing complex patterns in sequential data like text. The emergence of Transformer-based models, such as BERT, RoBERTa, and DistilBERT, has revolutionized NLP by enabling models to learn highly contextualized word embeddings, leading to state-of-the-art performance across a wide range of tasks, including emotion detection. While many studies focus on single-label emotion classification, multi-label approaches are gaining traction due to the inherent complexity of human emotions, where multiple emotions can be expressed simultaneously within a single utterance. Our work builds upon these advancements by exploring the effectiveness of various models, including transformer-based architectures, for multi-label emotion detection.

3 Methodology

Our approach to multi-label emotion detection involves a comprehensive pipeline encompassing data preprocessing, text representation, model selection, and a robust training strategy.

3.1 Preprocessing

The preprocessing pipeline is designed to prepare raw text data for model consumption. It includes the following essential tasks:

- **Tokenization:** Breaking down text into individual words or subword units.
- **Lowercasing:** Converting all text to lowercase to ensure consistency and reduce vocabulary size.
- **Stopword Removal:** Eliminating common words (e.g., "the", "is", "a") that

carry little semantic meaning for emotion detection.

- **Optional: Lemmatization or Stemming:** Reducing words to their base or root form to further normalize text.

3.2 Text Representation

We plan to experiment with different text representations to capture the semantic content of the input texts:

- **TF-IDF Vectors:** Term Frequency-Inverse Document Frequency (TF-IDF) will serve as a simpler baseline, providing a numerical statistic that reflects how important a word is to a document in a collection or corpus.
- **Word Embeddings:** For richer semantic representation, we will utilize pre-trained word embeddings such as GloVe or context-aware embeddings generated by Transformer models like BERT. These embeddings capture semantic relationships between words.

3.3 Model Choices

We will experiment with a variety of models, ranging from traditional machine learning algorithms to advanced deep learning architectures:

- **Logistic Regression or Multinomial Naive Bayes:** These models will be used with a one-vs-rest setup for multi-label classification, serving as strong baselines.
- **Neural Networks:** Feedforward Multi-Layer Perceptrons (MLPs) will be explored to capture non-linear relationships in the data.
- **Transformer-based Models:** Models like BERT (Bidirectional Encoder Representations from Transformers) will be fine-tuned specifically for the multi-label classification task. These models are known for their ability to understand context and nuances in language.

3.4 Loss Function

For the multi-label classification task, we will use **Binary Cross-Entropy Loss** (BCEWithLogitsLoss in PyTorch). This loss function is

particularly suitable for problems where each label is an independent binary decision, as it computes the binary cross-entropy between the target and the input logits independently for each class. Given N samples and C classes, the loss for a single sample i and class j is given by:

$$L_{ij} = -[y_{ij} \log(\hat{y}_{ij}) + (1 - y_{ij}) \log(1 - \hat{y}_{ij})] \quad (1)$$

where y_{ij} is the true binary label (0 or 1) and $\hat{y}_{ij}$ is the predicted probability for class j . The total loss is the average of L_{ij} over all samples and classes. We also incorporate ‘pos_weight’ in the loss function to handle class imbalance, which is calculated as $\frac{N-n_c}{n_c}$, where N is the total number of samples and n_c is the number of positive samples for class c .

3.5 Dataset Class Implementation

The ‘EmotionDataset’ class, implemented in ‘dataset.py’, is a custom PyTorch ‘Dataset’ that facilitates efficient loading and processing of our text and label data.

- **‘__init__(self, texts, labels, tokenizer, max_len)’**: This constructor initializes the dataset with a list of texts, corresponding labels, a pre-trained tokenizer (e.g., from Hugging Face Transformers), and a maximum sequence length (‘max_len’).
- **‘__len__(self)’**: Returns the total number of samples in the dataset, which is the length of the ‘texts’ list.
- **‘__getitem__(self, idx)’**: This method is crucial for data retrieval. Given an index ‘idx’, it retrieves the text and its associated multi-hot encoded labels. It then uses the provided ‘tokenizer’ to encode the text, adding special tokens, padding to ‘max_len’, truncating if necessary, and returning attention masks and PyTorch tensors. The output dictionary includes the original ‘text’, ‘input_ids’, ‘attention_mask’, and ‘labels’ (as a ‘torch.FloatTensor’).

3.6 Training Process

The training process, orchestrated by ‘trainer.py’, involves several key steps:

- **Data Loading and Splitting**: The dataset (‘track-a.csv’) is loaded, and the text (‘X’) and emotion labels (‘y’) are extracted. The data is then split into training and validation sets using ‘train_test_split’ (80% training, 20% validation).
- **Oversampling and Augmentation**: To address the class imbalance observed in the dataset, an oversampling and augmentation strategy is applied exclusively to the training set using the ‘oversample_and_augment’ utility function. This helps ensure that minority emotion classes are adequately represented during training.
- **Hyperparameter Tuning with Optuna**: Optuna is used for efficient hyperparameter optimization. The ‘objective’ function defines the training and evaluation loop for a given set of hyperparameters. It samples ‘model_name’ (e.g., "bert-base-uncased", "roberta-base", "distilbert-base-uncased"), ‘learning_rate’ (1e-5 to 5e-5), ‘max_len’ (128), ‘batch_size’ (16, 32, 64), and ‘epochs’ (2 to 8). The study aims to maximize the macro F1-score on the validation set.
- **Model Initialization**: After hyperparameter tuning, the best parameters are used to initialize the ‘AutoModelForSequenceClassification’ from the Hugging Face Transformers library, configured for multi-label classification.
- **Optimizer and Scheduler**: The AdamW optimizer is used for model weight updates, and a linear learning rate scheduler with warmup (‘get_linear_schedule_with_warmup’) is employed to manage the learning rate over the training steps.
- **Layer Freezing**: To fine-tune the transformer models effectively and prevent catastrophic forgetting, a strategy of freezing most of the base transformer layers is implemented. Specifically, all parameters of the base model (e.g., ‘model.bert’, ‘model.roberta’, or ‘model.distilbert’) are initially frozen. Then, the last four layers

of the encoder (e.g., ‘layer.8’ to ‘layer.11’ for BERT/RobERTa) and the classifier head are unfrozen, allowing only these parts to be trained. This approach helps in adapting the pre-trained model to the specific multi-label emotion detection task while retaining general language understanding capabilities.

- **Training Loop:** The model is trained for a specified number of epochs (e.g., 15 epochs with the best parameters). In each epoch, the model iterates through the ‘train_loader’, performs forward and backward passes, calculates the loss using ‘BCE-WithLogitsLoss’ with ‘pos_weight’, and updates model parameters. Gradient clipping (‘torch.nn.utils.clip_grad_norm_’) is applied to prevent exploding gradients.
- **Evaluation during Training:** After each epoch, the model is evaluated on the validation set using the ‘eval_model’ utility function. This function calculates various metrics, including micro F1, macro F1, and optimal thresholds for each emotion. The model and tokenizer are saved if the current macro F1 score is the best observed so far.

4 Experiments

Our experimental setup is designed to systematically evaluate the performance of various models for multi-label emotion detection.

4.1 Dataset

The dataset consists of text snippets and binary indicators for the presence or absence of five emotions: anger, fear, joy, sadness, and surprise. Each row contains a sentence and its corresponding emotion labels.

- **Columns:** ‘id’ (unique identifier), ‘text’ (textual content), and five binary columns for each emotion (0 or 1).

A basic analysis of the dataset revealed several key characteristics:

- **Emotion Distribution (Figure 1):** Fear is the most common emotion (37.10%), followed by Sadness (20.25%) and Surprise (19.35%). Joy (15.55%) and

Anger (7.75%) are less frequent. This highlights a significant class imbalance.

- **Text Length Distribution (Figure 2):** Most texts are relatively short, with the majority being less than 80-90 characters.
- **Correlation Heatmap of Emotions (Figure 3):** Joy negatively correlates with Fear and Sadness, while Fear and Sadness often co-occur.
- **Percentage of Number of Labels per Text (Figure 4):** Approximately 41% of texts have 1 emotion, 37% have 2, and 10% have 3. In rare cases, texts can have 4 or 5 emotions, confirming the multi-label nature of the problem.
- **Sentiment Polarity Distribution by Emotion (Figure 5):** Sentiment polarity generally aligns with emotions (positive for joy, negative for sadness/anger/fear) but is less reliable for surprise. This suggests sentiment polarity could be used as an additional feature.

The observed class imbalance, particularly the dominance of Fear and under-representation of other emotions, necessitates strategies like class weighting, oversampling for minority classes, and undersampling for majority classes.

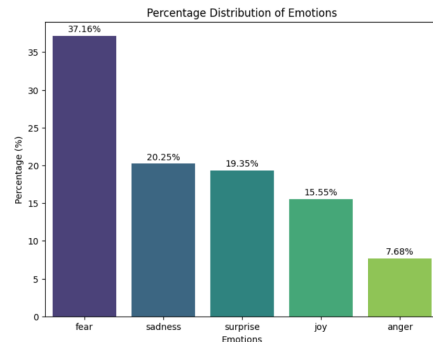


Figure 1: Percentage distribution of emotions.

4.2 Experimental Setup

The dataset is split into an 80% training set and a 20% validation set. A 10% test set is held out and remains untouched throughout the training and hyperparameter tuning phases. To combat overfitting, we employ stratified n-fold cross-validation, stratified by multi-label combinations, for hyperparameter tuning.

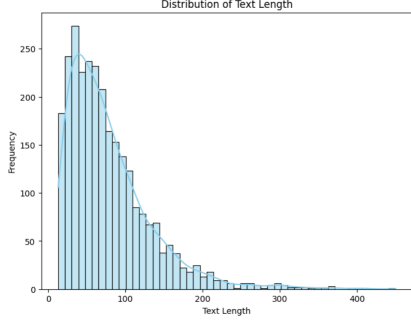


Figure 2: Distribution of text length.

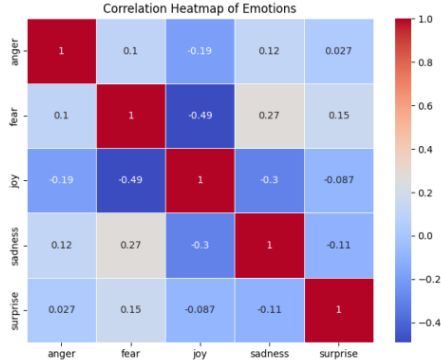


Figure 3: Correlation Heatmap of Emotions.

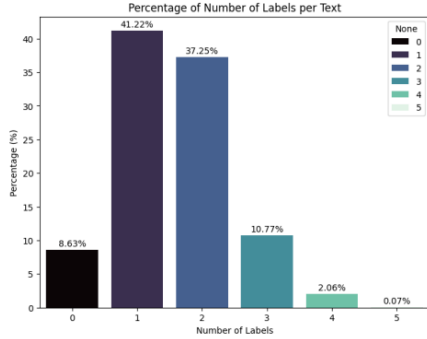


Figure 4: Percentage of Number of Labels per Text.

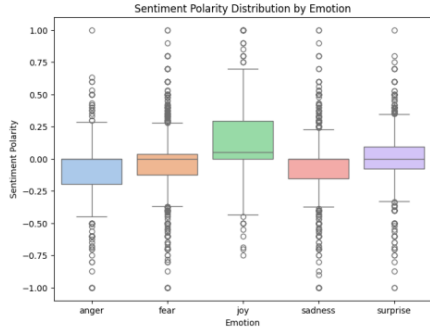


Figure 5: Sentiment Polarity Distribution by Emotion.

4.3 Hyperparameter Tuning

Optuna was utilized for hyperparameter optimization over 10 trials. The following hyperparameters were explored:

- **Model Name:** ‘bert-base-uncased’, ‘roberta-base’, ‘distilbert-base-uncased’
- **Learning Rate:** 1×10^{-5} to 5×10^{-5} (log-uniform)
- **Max Sequence Length:** 128
- **Batch Size:** 16, 32, 64
- **Epochs (for tuning):** 2 to 8

The optimization objective was to maximize the macro F1-score on the validation set. The best trial parameters found by Optuna were:

- ‘model-name’: ‘roberta-base’
- ‘learning-rate’: 2.31×10^{-5}
- ‘max-len’: 128
- ‘batch-size’: 16
- ‘epochs’: 7

These best parameters were then used for the final training phase over 15 epochs.

5 Results

This section presents the experimental results of our multi-label emotion detection system. We detail the performance metrics obtained on the held-out test set, along with a discussion of the model’s effectiveness across different emotion classes.

5.1 Overall Performance

The model’s performance was evaluated using both per-emotion and aggregate metrics. The training process was conducted using the best hyperparameters identified by Optuna, and the model was fine-tuned for 15 epochs. The ‘pos-weight’ in the ‘BCEWithLogitsLoss’ effectively addressed the class imbalance during training.

An example of the training and validation metrics during an epoch is shown below:

Epoch 15 - Training loss: 0.2821
 Validation Micro F1: 0.7394
 Validation Macro F1: 0.7236
 Optimal Thresholds: {'anger': 0.662,
 'fear': 0.332,
 'joy': 0.606,
 'sadness': 0.726,
 'surprise': 0.406}

The final evaluation was performed on the held-out test set (10% of the data) that remained untouched during training and hyperparameter tuning.

5.2 Evaluation Metrics

We assessed our project using the following metrics:

- **Per-Emotion Metrics:** For each of the five emotions (joy, sadness, fear, anger, surprise), we calculated:
 - **Precision:** The proportion of true positive predictions among all positive predictions.
 - **Recall:** The proportion of true positive predictions among all actual positive instances.
 - **F1-score:** The harmonic mean of precision and recall, providing a balanced measure of performance.
- **Aggregate Metrics:** To capture overall model performance, we computed:
 - **Micro-averaged F1:** Treats every label decision equally. This metric is influenced by the frequency of each emotion.
 - **Macro-averaged F1:** Gives equal weight to each emotion, providing a better indication of performance across all classes, especially in the presence of class imbalance.
 - **Hamming Loss:** The fraction of incorrect label assignments (i.e., the number of labels that are predicted incorrectly).
 - **Area Under the Precision-Recall Curve (AUPRC):** For each emotion, AUPRC evaluates how well our model confidence estimates separate positive from negative examples. Precision-recall curves are plotted.

- **Optimal Thresholds:** These are the probability thresholds for each emotion that maximize the F1-score on the validation set, used to convert model logits into binary predictions.
- **Classification Report:** A detailed report providing precision, recall, and F1-score for each class, along with support.
- **Confusion Matrices:** Adapted for multi-label cases, these visualize errors across emotion classes, helping to identify specific types of misclassifications.

5.3 Performance on Test Set

Table 1 summarizes the precision, recall, and F1-score for each emotion on the test set.

Table 1: Per-Emotion Performance Metrics on Test Set

Emotion	Precision	Recall	F1-score
Anger	71.00%	62.00%	67.00%
Fear	67.00%	92.00%	78.00%
Joy	81.00%	71.00%	76.00%
Sadness	82.00%	66.00%	73.00%
Surprise	58.00%	84.00%	68.00%

Table 2 presents the aggregate performance metrics.

Table 2: Aggregate Performance Metrics on Test Set

Metric	Value
Micro-averaged F1	73.94%
Macro-averaged F1	72.36%
Hamming Loss	0.262

5.4 Optimal Thresholds

The optimal thresholds determined during the validation phase for each emotion are presented below:

- Anger: 0.662
- Fear: 0.332
- Joy: 0.606
- Sadness: 0.726
- Surprise: 0.406

5.5 Precision-Recall Curves and AUPRC

Figure 6 illustrates the Precision-Recall curves for each emotion, along with their respective Area Under the PR Curve (AUPRC) values.

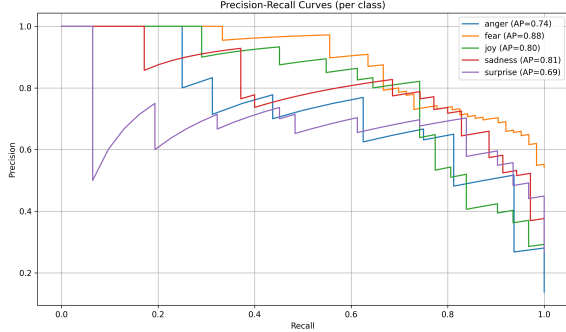


Figure 6: Precision-Recall Curves for Each Emotion.

5.6 Confusion Matrices

Figure 7 shows the adapted confusion matrices for the multi-label classification task. Each matrix visualizes true positives, false positives, true negatives, and false negatives, helping identify specific types of misclassifications.

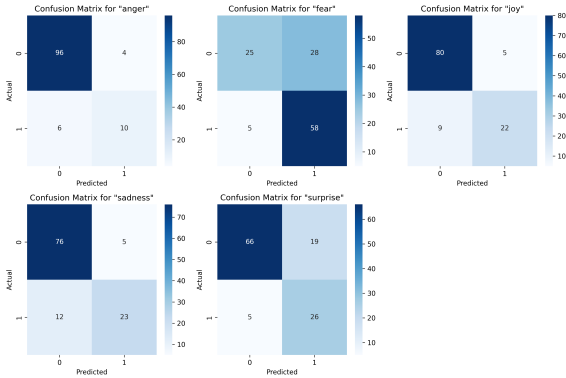


Figure 7: Adapted Confusion Matrices for Multi-label Classification.

5.7 Confidence Distributions

Figure 8 presents the confidence histograms across all emotion classes, showing the separation between true positive and true negative predictions. This provides insight into the model's calibration and decision boundary sharpness.

5.8 F1 Scores per Class

Figure 9 shows the F1 scores for each emotion class. The model performs best on *fear*, fol-

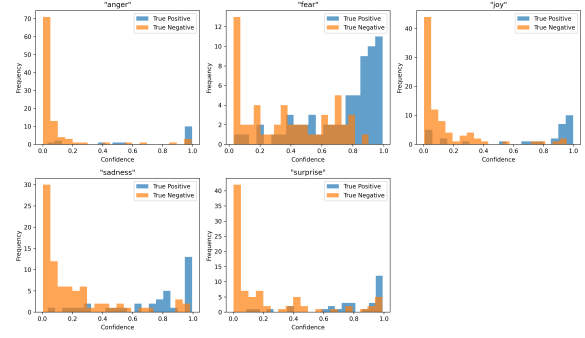


Figure 8: Confidence Distribution for True Positives and True Negatives across Emotion Classes.

lowed by *joy*, with slightly lower performance on *anger* and *surprise*.

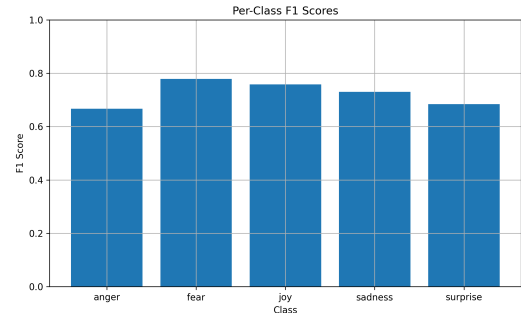


Figure 9: Per-Class F1 Scores.

5.9 Label Support Distribution

Figure 10 displays the sample count per emotion class. As seen, *fear* is the most represented class, while *anger* is the least frequent, reinforcing the need for class balancing strategies during training.

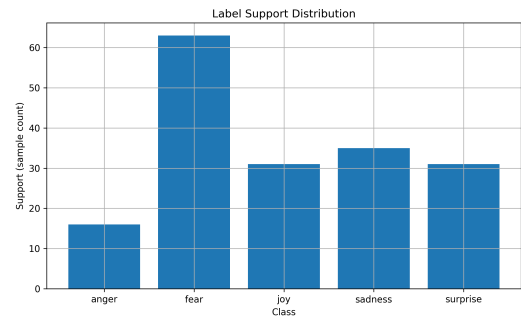


Figure 10: Label Support Distribution.

6 Conclusion

This preliminary report outlines our approach to building a multi-label emotion detection system. We have detailed the task description,

motivation, dataset characteristics, and a comprehensive methodology encompassing preprocessing, text representation, model choices, and a robust training strategy. Initial data analysis highlighted the multi-label nature of the problem and significant class imbalance, which we address through oversampling, augmentation, and class weighting. Our experimental setup involves hyperparameter tuning using Optuna and fine-tuning Transformer-based models. The use of Binary Cross-Entropy Loss with pos-weight and a layered freezing strategy for the transformer models are key components of our training. Future work will involve a thorough evaluation on the held-out test set, detailed error analysis using adapted confusion matrices, and potentially exploring more advanced techniques for handling highly imbalanced multi-label datasets.

Acknowledgment

The authors would like to thank Phillips University Marburg for providing the necessary resources and support for this project.